Image Processing with OpenCV

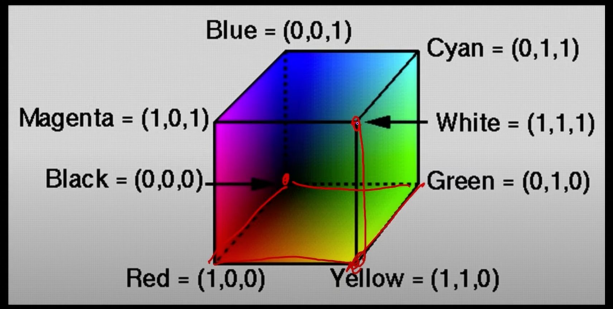
AIM825: Visual Recognition

Color Theory

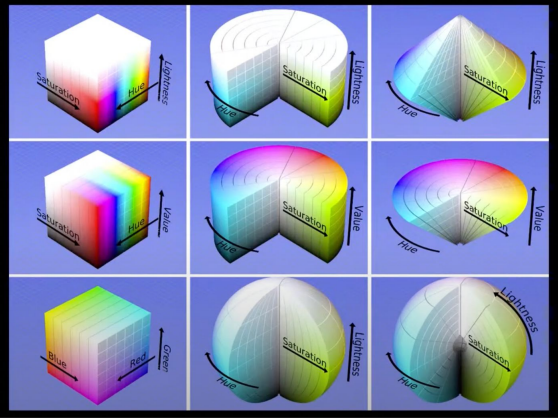
Human Response to different wavelengths

Visible light:

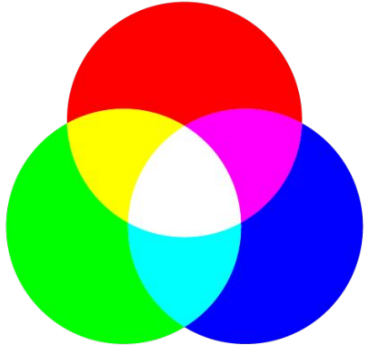
- RGB color space



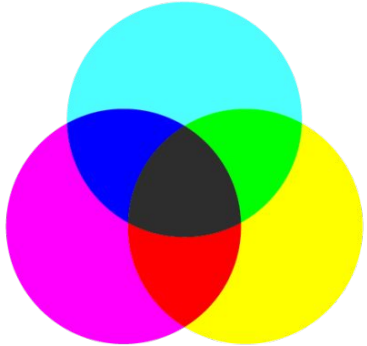
But there are lots of color spaces



Additive colour



Subtractive colour



Changing color spaces

```
>>> import cv2 as cv
>>> flags = [i for i in dir(cv) if i.startswith('COLOR_')] #Get all required color codes
>>> print( flags )
```

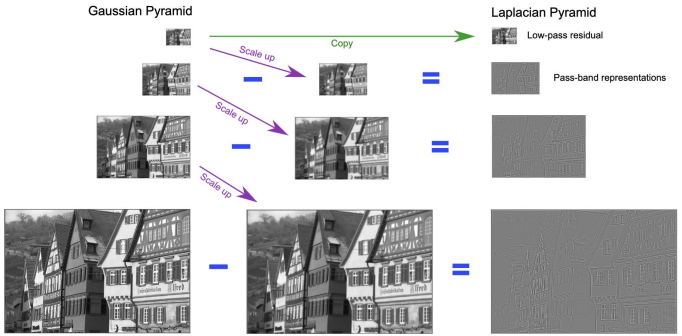
- Note that OpenCV reads channels in BGR

```
hsv = cv.cvtColor(frame, cv.COLOR_BGR2HSV)
```

- This is how you should do it,
- Applications of color spaces - Specific Object Tracking which has a very distinguishable color in one of the spaces from other objects on the scene.

Downsampling and Upsampling

1. **pyrUp()** and **pyrDown()** are the required methods
2. An image pyramid is a collection of images - all arising from a single original image - that are successively downsampled until some desired stopping point is reached.
3. There are two common kinds of image pyramids:
   a. **Gaussian pyramid:** Used to downsample images
   b. **Laplacian pyramid:** Used to reconstruct an upsampled image from an image lower in the pyramid (with less resolution)
4. For downsampling you convolve with a Gaussian Pyramid (Convolutions from Signals YES.)
5. The fundamental idea behind Laplacian pyramids is to represent an image as a set of **increasingly coarse approximations** along with their corresponding **detailed differences** (residuals). This enables efficient image compression, blending, and reconstruction.



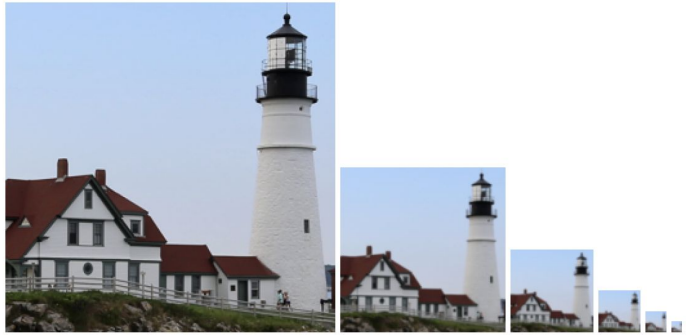


Figure 3.2: Gaussian pyramid. The Gaussian pyramid is built for each color channel independently.

Histogram Equalization

- Consider histogram as a graph or plot, which gives you an overall idea about the intensity distribution of an image. It is a plot with pixel values (ranging from 0 to 255, not always) in X-axis and corresponding number of pixels in the image on Y-axis
- Gain intuition about contrast, brightness, intensity distribution
- So now we use **cv.calcHist()** function to find the histogram. Let's familiarize with the function and its parameters :
    - *cv.calcHist(images, channels, mask, histSize, ranges[, hist[, accumulate]])*

    1. images : it is the source image of type uint8 or float32. it should be given in square brackets, ie, "[img]".
    2. channels : it is also given in square brackets. It is the index of channel for which we calculate histogram. For example, if input is grayscale image, its value is [0]. For color image, you can pass [0], [1] or [2] to calculate histogram of blue, green or red channel respectively.
    3. mask : mask image. To find histogram of full image, it is given as "None". But if you want to find histogram of particular region of image, you have to create a mask image for that and give it as mask.
    4. histSize : this represents our BIN count. Need to be given in square brackets. For full scale, we pass [256].
    5. ranges : this is our RANGE. Normally, it is [0,256].

Now **Histrogram Equaization redistributes pixel intensities** so that the histogram (distribution of intensity levels) becomes more uniform, enhancing overall contrast.

Image Filters

- Image blurring is achieved by convolving the image with a low-pass filter kernel. It is useful for removing noise. It actually removes high frequency content (eg: noise, edges) from the image. So edges are blurred a little bit in this operation (there are also blurring techniques which don't blur the edges). OpenCV provides four main types of blurring techniques.
  - ***Averaging*** - Computes the mean of neighboring pixels, producing a simple smoothing effect but can introduce artifacts.
  - ***Gaussian Blurring*** - Applies a weighted Gaussian kernel for natural, edge-preserving smoothing, reducing noise effectively.
  - ***Median Blurring*** - Replaces each pixel with the median of surrounding pixels, excellent for removing salt-and-pepper noise.
  - ***Bilateral Filtering*** - Preserves edges while smoothing by considering both spatial and intensity similarities.

Fourier Transform

- The Fourier Transform will decompose an image into its sinus and cosines components. In other words, it will transform an image from its spatial domain to its frequency domain.
- The idea is that any function may be approximated exactly with the sum of infinite sinus and cosines functions.
- Throughout the image processing algorithms only the magnitude image is interesting as this contains all the information we need about the images geometric structure
- Why are we using dft for images?

Gradient-based Edge detection

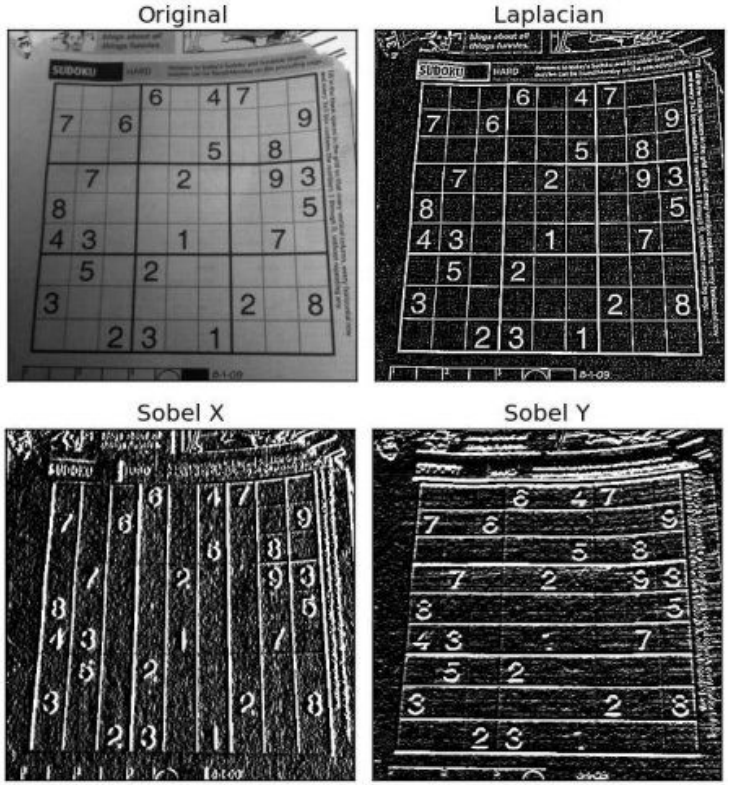
Sobel Operator: Computes image gradients using convolution, emphasizing edges in both horizontal and vertical directions.

Prewitt Operator: Similar to Sobel but uses a simpler kernel, detecting edges based on intensity changes.

Gradient Magnitude: Represents edge strength by combining horizontal and vertical gradients.

Gradient Direction: Determines edge orientation, useful for contour analysis and feature extraction.

Sobel operators is a joint Gaussian smoothing plus differentiation operation, so it is more resistant to noise



Canny Edge Detection

1. Noise Reduction

- Apply a Gaussian filter to smooth the image, reducing noise and minor details.

2. Gradient Calculation

- Compute the intensity gradients using Sobel operators to detect edge strength and direction.

3. Non-Maximum Suppression

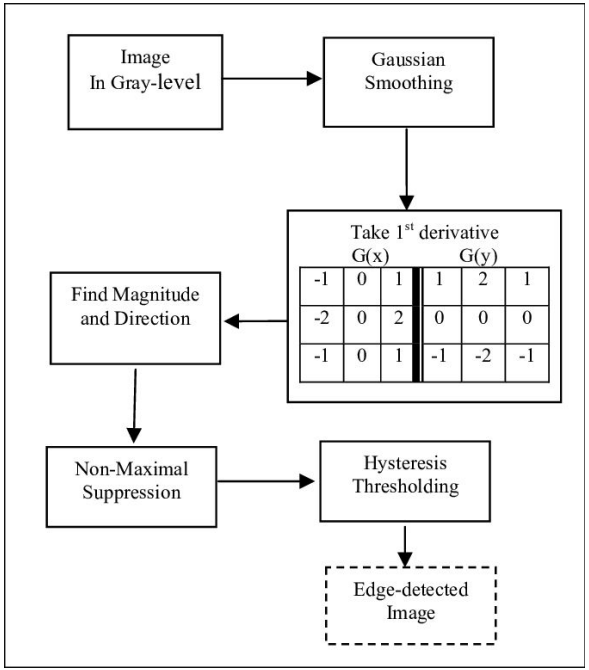
- Thin out the edges by retaining only local maxima in the gradient direction, eliminating non-maximum pixels.

4. Double Thresholding

- Classify pixels as strong, weak, or non-edges based on high and low threshold values.

5. Edge Tracking by Hysteresis

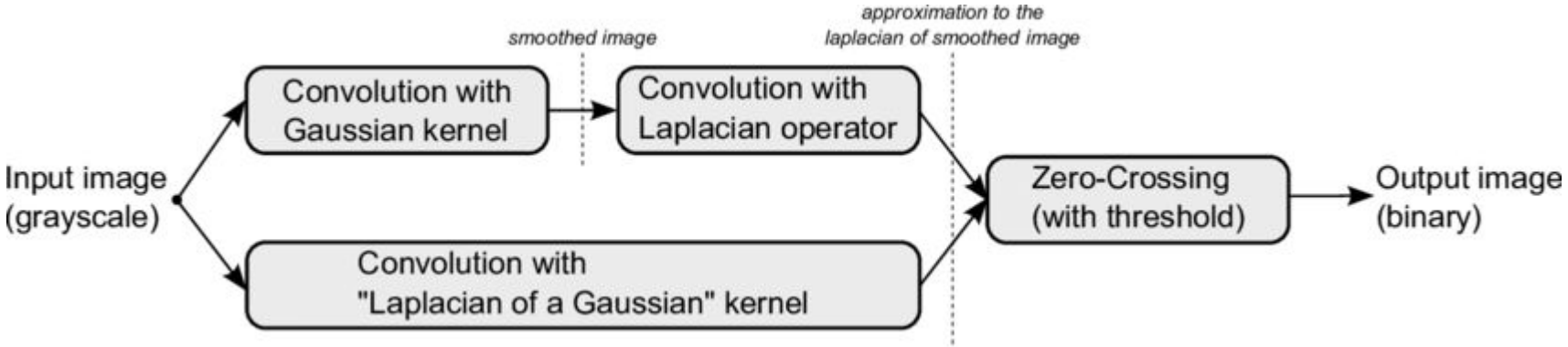
- Finalize edge detection by connecting weak edge pixels that are adjacent to strong ones, ensuring continuity.



Marr-Hildreth Edge Detection

```
# Apply Marr-Hildreth edge detection using LoG filter

laplacian = cv2.Laplacian(gray_image, cv2.CV_64F)

laplacian_abs = np.absolute(laplacian)

marr_hildreth_e = cv2.convertScaleAbs(laplacian_abs)

# Perform the element-wise addition for Marr-Hildreth
g_marr_hildreth = cv2.add(f, cv2.merge([marr_hildreth_e] * 3) * c)
g_marr_hildreth = np.clip(g_marr_hildreth, 0, 255).astype(np.uint8)
```

- Apply a Gaussian filter to the image to reduce noise and minor variations, ensuring that edge detection focuses on significant structures.
- Compute the Laplacian of the smoothed image to highlight regions of rapid intensity change, which correspond to potential edges.
- Identify points in the LoG-filtered image where the intensity changes sign (i.e., crosses zero), as these zero-crossings indicate the locations of edges.



Harris Corner Detector:

- Corners are regions in the image with large variation in intensity in all the directions
- Core concept - Basically find the difference in intensity for a displacement of in all directions using a rectangular window or gaussian window.
- OpenCV has the function **cv.cornerHarris()** for this purpose. Its arguments are:
  - **img** - Input image. It should be grayscale and float32 type.
  - **blockSize** - It is the size of neighbourhood considered for corner detection
  - **ksize** - Aperture parameter of the Sobel derivative used.
  - **k** - Harris detector free parameter in the equation.

Scale-Invariant Feature Transform (SIFT):

Harris detector is rotation-invariant, which means, even if the image is rotated, we can find the same corners. It is obvious because corners remain corners in rotated image also.

But what about scaling? A corner may not be a corner if the image is scaled. A corner in a small image within a small window is flat when it is zoomed in the same window. So Harris corner is not scale invariant.

Key Steps:

1. **Scale-Space Extrema Detection**: Identifies potential keypoints by detecting local extrema in the Difference of Gaussians across various scales.
2. **Keypoint Localization**: Refines keypoint positions and filters out those with low contrast or poorly localized along edges.
3. **Orientation Assignment**: Assigns consistent orientations to keypoints based on local image gradient directions.
4. **Keypoint Descriptor Generation**: Creates descriptors by analyzing local gradients around each keypoint, typically resulting in 128-dimensional vectors.

```
# In OpenCV
sift = cv.SIFT_create()
kp, des = sift.detectAndCompute(gray,None)
```